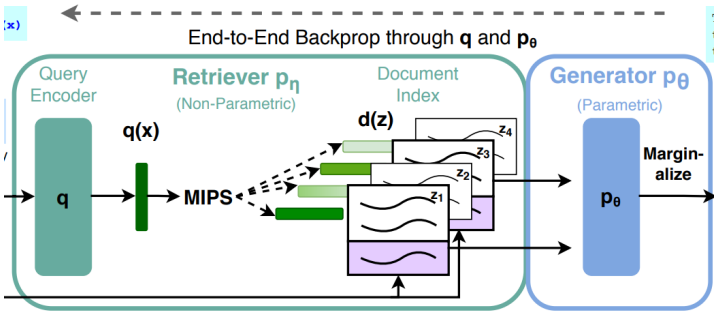
Deep Learning for NLP II

Retrieval-Augmented Generation (RAG)



Learning goals

- Understand RAG pipelines
- Overview of retrieval methods
- Understand RAG evaluation
- Practical considerations

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

MOTIVATION

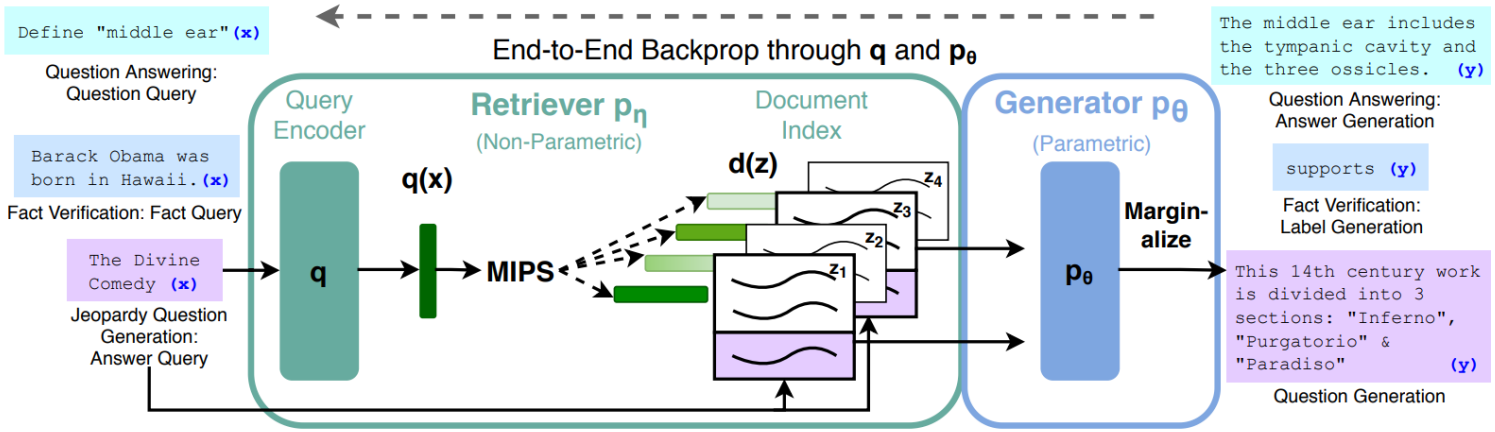
Motivation:

- LLMs can do **amazing** things – but they are prone to hallucinations
- Many schemes of types of hallucinations exist ▸ Huang et al. (2025) [8]
- Examples:
  - *Factuality hallucinations* contradicting facts
  - *Faithfulness hallucinations* lacking logical consistence
- Real-world examples:
  - Lawyers cited fake ChatGPT cases ▸ Reuters (2023)
  - Air Canada chatbot hallucinated refund policy
    $\rightarrow$ lost lawsuit ▸ CBC (2024)

Idea of RAG: Provide relevant external knowledge at inference time

THE ORIGINAL RAG PAPER

Setup: Combine a *parametric* memory (seq2seq generator) with a *non-parametric* memory (dense document index) ▸ Lewis et al. (2020) [12]



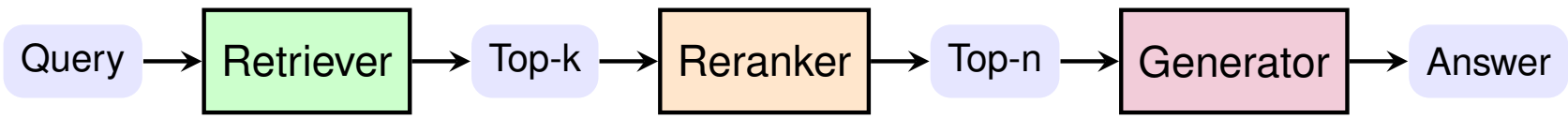
▸ Source: [12]

Component	Choice
Retrieval	Embedding-based (separate `BERT` models for q and d)
Document index	21M Wikipedia passages
Generator	`BART-large` (400M params)
Training	**End-to-end, no retrieval supervision**

RETRIEVAL-AUGMENTED GENERATION

Core idea/design choices:

- Retrieve relevant documents ("*evidence*"), then generate answers grounded in this evidence
- Use increasingly strong (and expensive) models at each stage
- Example of typical 2-stage retrieval (retriever + reranker)



Component	Latency	What it optimizes
Retriever (sparse vs. dense)	100-300ms	Recall (M $\rightarrow$ 100s)
Reranker	100-2000ms	Precision (100s $\rightarrow$ 10s)
Generator (LLM)	1000+ms	Quality

DOCUMENT CHUNKING

From Documents to Retrievable Units: Prior to retrieval we need document **chunks**, i.e., *meaningful semantic units*

Why chunk?

- Long documents (100+ pages) consume too much LLM context
- Enables more precise retrieval of relevant passages

Trade-offs:

Size	Pros	Cons
Smaller	Precise matching	Loses context
Larger	Preserves context	Adds noise

DOCUMENT CHUNKING: STRATEGIES (1)

Fixed-size splitting (baseline)

- Split every *n* tokens, optionally with overlap
- Fast, but may cut mid-sentence or mid-argument

Recursive splitting ▸ LangChain docs

- Try splitting on "`\n\n`" first, then "`\n`", then spaces – falling back to the next separator only if chunks are still too large
- Naturally respects paragraphs $>$ sentences $>$ words; Recommended in LangChain

Sentence-boundary-aware splitting

- Respect sentence boundaries (e.g. via spaCy / NLTK sentence segmentation)
- Chunks stay linguistically coherent; embeddings are more meaningful (compared to fixed-size splitting)

DOCUMENT CHUNKING: STRATEGIES (2)

Semantic chunking

- Embed consecutive sentences; insert a boundary wherever cosine similarity *drops* (more than a predefined theshold)
- Groups sentences that talk about the same topic – boundaries follow content, not word count

Parent-document retriever ▸ LangChain docs

- Index *small* child chunks for precise retrieval . . .
- . . . but return the *larger* parent passage to the generator
- *Best of both worlds:*
  - fine-grained matching
  - sufficient context for generation

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

RETRIEVAL METHODS: OVERVIEW

Three main pillars:

BM25 (Sparse)	SBERT/DPR (Dense)	Combinations (Hybrid)
statistical	text embedding	merges rankings

Key insight:
No single best method – tradeoffs between speed, accuracy and interpretability

BM25: BEST MATCH 25

Representation: BM25 represents documents as sparse vectors (one dimension per vocabulary term) ▸ Robertson & Zaragoza (2009) [19]

Three intuitions:

1. **Rare terms matter more** (IDF): "transformer" $>$ "the"
2. **Diminishing returns** (TF saturation):
   saying "good" $10\times \neq 10\times$ more relevant
3. **Length normalization**: longer docs not automatically better

Strengths	Limitations
Fast implementation	Vocabulary mismatch ("car" $\neq$ "automobile")
Interpretable	No semantic understanding

BM25: BEST MATCH 25 – FORMULA (1)

Scoring function (for query $Q = \{q_1, \ldots, q_n\}$ and document D):

$$\text{BM25}(D, Q) = \sum_{i=1}^{n} \underbrace{\text{IDF}(q_i)}_{\text{term weight}} \cdot \underbrace{\frac{f(q_i, D)\,(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b\,\frac{|D|}{\text{avgdl}}\right)}}_{\text{term frequency (saturated \& length-normalised)}}$$

Component breakdown:

- $f(q_i, D)$: raw term frequency of q_i in D
- $|D|$ / avgdl: document length relative to corpus average
  $\rightarrow$ prevents long documents from being favoured by chance
- $k_1 \in [1.2, 2.0]$: **TF saturation** – controls how fast the TF contribution plateaus (e.g. $k_1 = 0 \Rightarrow$ pure IDF; $k_1 \rightarrow \infty \Rightarrow$ raw TF)

BM25: BEST MATCH 25 – FORMULA (2)

Scoring function (for query $Q = \{q_1, \ldots, q_n\}$ and document D):

$$\text{BM25}(D, Q) = \sum_{i=1}^{n} \underbrace{\text{IDF}(q_i)}_{\text{term weight}} \cdot \underbrace{\frac{f(q_i, D)\,(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b\,\frac{|D|}{\text{avgdl}}\right)}}_{\text{term frequency (saturated \& length-normalised)}}$$

Component breakdown:

- $b \in [0, 1]$: **length normalization strength**
  (b=0: no normalization, b=1: full normalization)
- $\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$: inverse document frequency
  (N = corpus size, $n(q_i)$ = number of docs containing q_i)

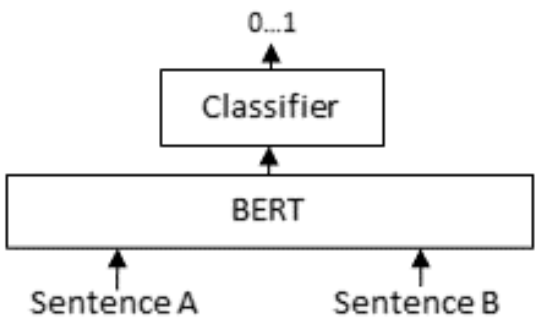
Defaults: $k_1 = 1.5$, $b = 0.75$ work well across many collections

FROM STATISTICAL TO EMBEDDING-BASED (1)

Remember: Word embeddings (e.g., word2vec) give fixed vectors per word, but do not carry sentence-level meaning

Initial idea: Use BERT to classify sentence similarity (**cross-encoder**)

`[CLS] sent A [SEP] sent B [SEP]` $\rightarrow$ BERT $\rightarrow$ *similar / not similar*



▸ Source: SBERT homepage

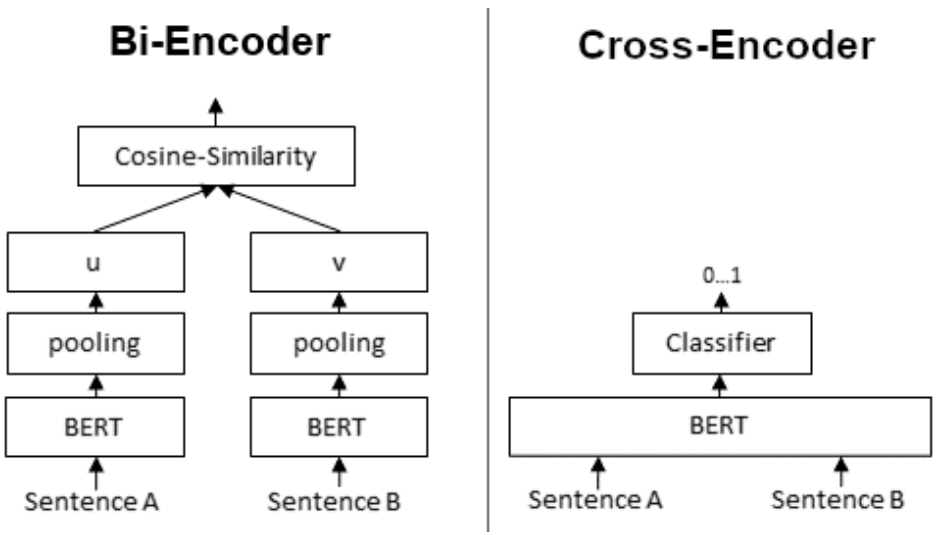
FROM STATISTICAL TO EMBEDDING-BASED (2)

Problem: Requires $O(n^2)$ comparisons!

- Finding most similar pair in 10,000 sentences = 50M comparisons (∼65 hours!) ▸ Reimers & Gurevych (2019) [18]
- Not suitable for retrieval at scale

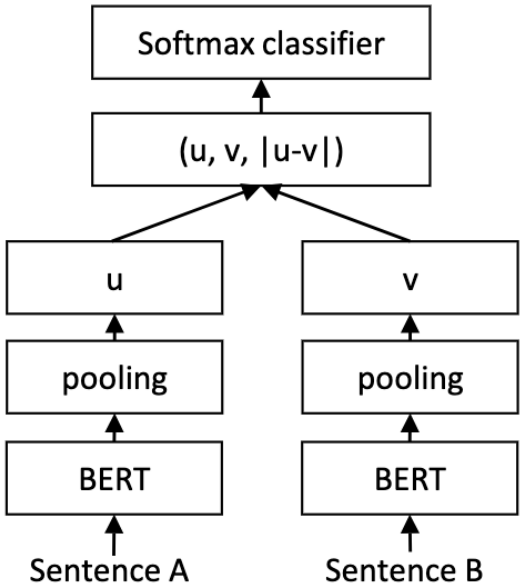
Idea: Learn fixed-size sentence embeddings that can be pre-computed



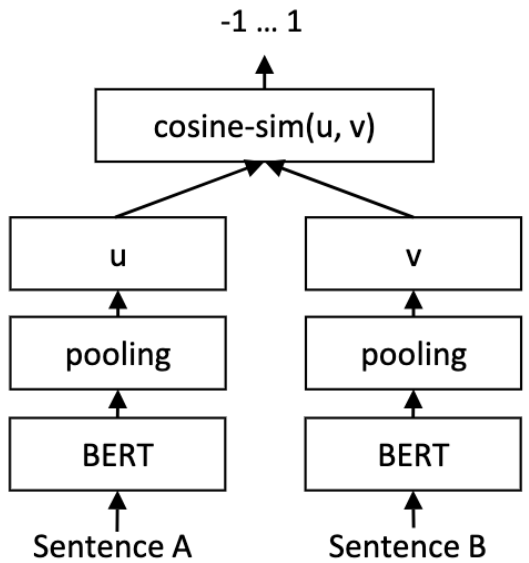


▸ Source: SBERT homepage

SENTENCE-BERT (SBERT)



Training (Shared weights)



Inference (Individual Encoders)

▸ Source: Reimers & Gurevych (2019) [18]

Key Insight: Shared BERT weights $\rightarrow$ comparable embedding space.

- **Symmetric search:** Find text that *means the same thing*.
- **Training** nn NLI and Sentence Similarity Tasks

Impact: 10K comparisons reduced from 65 hours $\rightarrow$ **5 seconds**

SBERT: ASYMMETRY PROBLEM

Problem: SBERT is trained on general similarity, i.e., similar questions have similar embeddings and similar documents have similar embeddings.

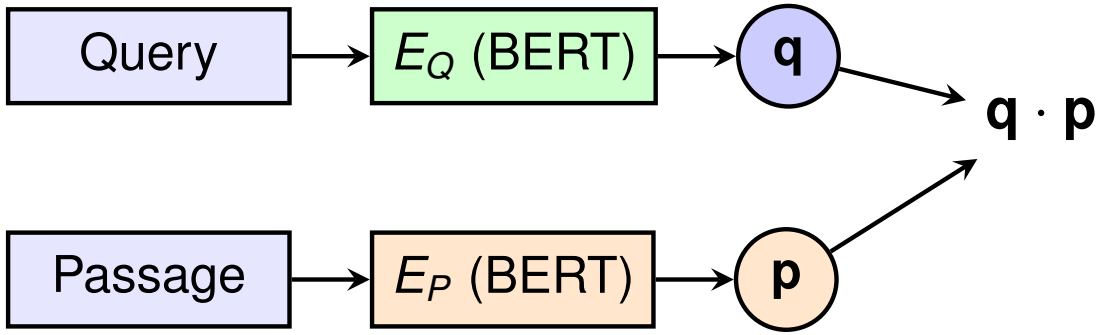
$$\textit{questions} \neq \textit{documents}$$

$\Rightarrow$ Retrieval might fail due to this asymmetry

Idea: Train asymmetric embeddings, i.e. train whether an passage answers a question.

Dense Passage Retrieval (DPR) ▸ Karpukhin et al. (2020) [10]

DPR (1): OVERVIEW



$E_Q \neq E_P$ (different params)

SBERT (Symmetric)	**DPR (Asymmetric)**
Same encoder (shared)	Different encoders ($E_Q \neq E_P$)
Trained on NLI/STS	Trained on QA pairs

DPR (2): TRAINING & RETRIEVAL

Training objective: Maximize similarity of positive (question, passage) pairs and minimize it for negatives

$$L = -\log \frac{e^{\mathbf{q}^\top \mathbf{p}^+}}{e^{\mathbf{q}^\top \mathbf{p}^+} + \sum_{j=1}^{m} e^{\mathbf{q}^\top \mathbf{p}_j^-}}$$

- $\mathbf{q} = E_Q([\text{CLS}])$, $\mathbf{p} = E_P([\text{CLS}])$ – the [CLS] vectors of the respective BERT encoders
- Hard negatives: top BM25 passages that do *not* answer the question $\rightarrow$ much more effective than random negatives
- In-batch negatives: other passages in the same mini-batch serve as cheap negatives

DPR (2): TRAINING & RETRIEVAL

Inference:

1. **Offline:** Encode all M passages once with $E_P \rightarrow$ store vectors
2. **Online:** Encode query with $E_Q \rightarrow$ retrieve top-k by dot product

Results:

DPR outperforms BM25 by $>$9% EM on Natural Questions

▸ Karpukhin et al. (2020) [10]

EFFICIENT SIMILARITY SEARCH

Problem: After encoding millions of passages, brute-force search is still slow.

FAISS (Facebook AI Similarity Search): ▸ Douze et al. (2024) [3]

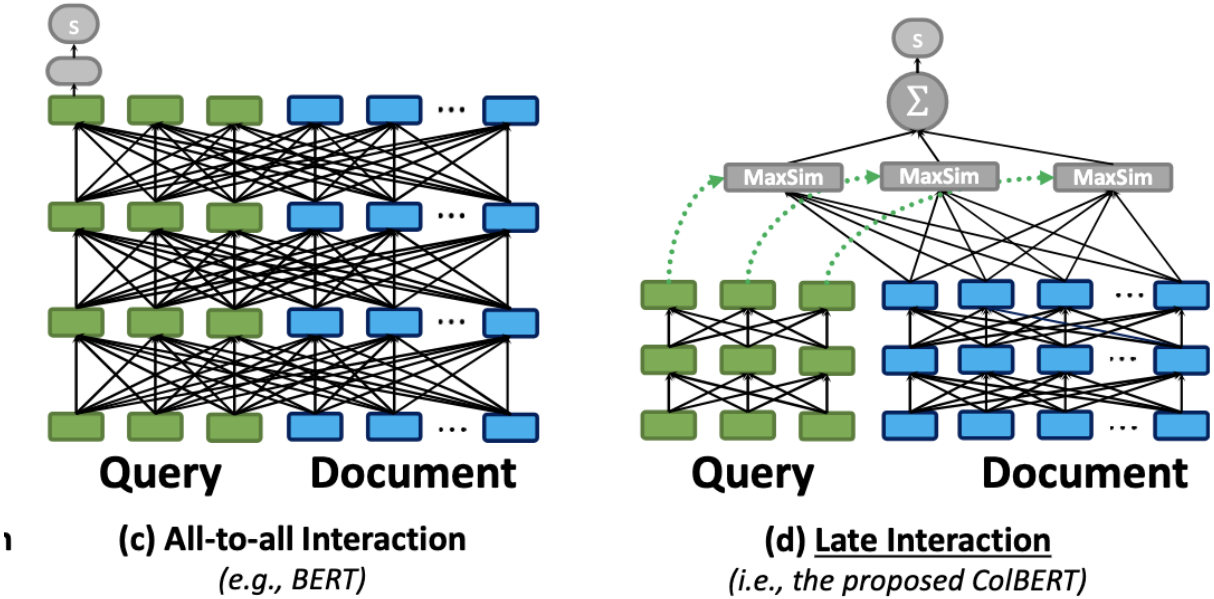
- Approximate Nearest Neighbor (ANN) search
- Trades exact results for massive speed gains
- DPR + FAISS searches 21M passages in **milliseconds**

Method	**Speed**	**Recall**	**Memory**
Flat (exact)	Slow	100%	High
IVF	Fast	∼95%	Medium
IVF+PQ	Very fast	∼90%	Low

Key insight: Similarity search is **solved infrastructure** – use FAISS, Pinecone, Weaviate, . . .

COLBERT: LATE INTERACTION

Problem: Single-vector embeddings lose information
Idea: Preserve per-token representations



*__MaxSim score:__ For each query token, find best-matching doc token, then sum scores

▸ Source: [11]

COLBERT: LATE INTERACTION

Comparison to cross-encoders:

	Cross-Encoder	Late Interaction (ColBERT) ▸ Khattab & Zaharia (2020) [11]
Encoding	Joint (query + doc together)	Separate (pre-compute docs)
Interaction	Full cross-attention	MaxSim at retrieval
Speed	Slow (scoring only)	Fast (can retrieve)
Quality	Good	Close to cross-encoder

Why ColBERT matters:

- Pre-computes *all* document token vectors offline
  $\rightarrow$ fast at query time
- MaxSim aggregation retains fine-grained token-level matching without cross-attention overhead
- *Scales to millions of documents*: ColBERTv2 ▸ Santhanam et al. (2022) [21] adds compression (residual quantization) to reduce storage $\sim$6–10$\times$

RETRIEVAL QUALITY BOUNDS GENERATION QUALITY

Principle: “Garbage in, garbage out”
$\rightarrow$ If the retriever misses relevant documents, the generator cannot recover. Most RAG failures are retrieval failures.

There are many issues and biases inherent to RAG systems. Examples of embedding biases decreasing performance: ▸ Fayyaz et al. (2025) [5]

- **Short Doc Bias:** Prefers shorter passages.
- **Position Bias:** Favors early content.
- **Literal Match:** Over-weights exact terms.

Comprehensive RAG Survey: ▸ Gao et al. (2023) [7]

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

THE CASE FOR RERANKING

Division of labor:

- *Retriever*: Must be fast (search millions) $\rightarrow$ optimizes recall
- *Reranker*: Can be slower (score $\sim$100s) $\rightarrow$ optimizes precision

Typical setup:

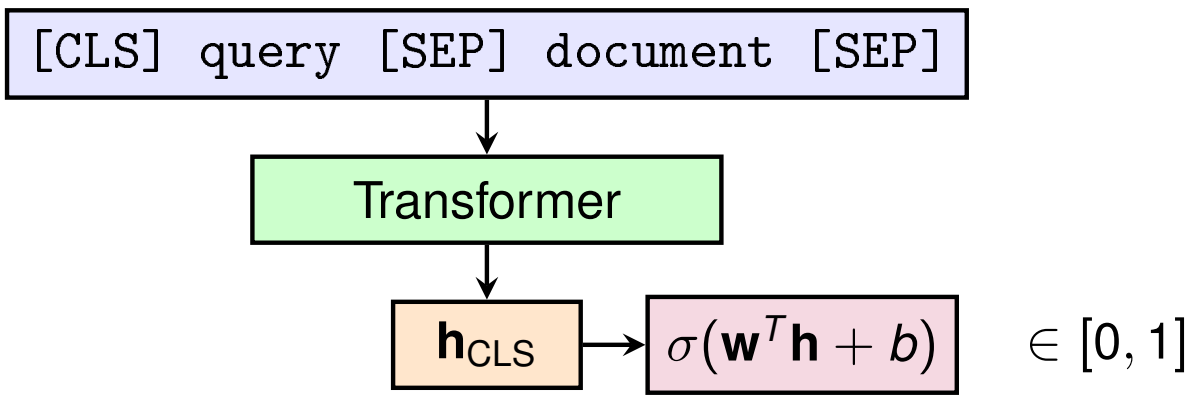
Retrieve 100-1000 docs $\rightarrow$ Rerank $\rightarrow$ Keep top 5-20

Cost-benefit:

- Reranking adds $\sim$1-2 seconds but dramatically improves quality
- “Cheap insurance” for RAG systems

CROSS-ENCODERS FOR RERANKING

Key insight: Joint encoding enables **full cross-attention** between query and document tokens ▸ Nogueira & Cho (2019) [17]



Benefits:

- Full information: Joint (query + doc together)
- No cosine similarity, full cross-attention

LLM-BASED RERANKING

Motivation: LLMs for Re-Ranking ▸ Sun et al. (2023) [23] ▸ Abdallah et al. (2025) [1]

Pointwise – Independent scoring of documents:

- Prompt: "*Rate relevance of this document to the query (1-5)*"
- Simple, parallelizable, but no cross-document comparison

Listwise – Rank all documents together:

- Prompt: "Rank documents [A], [B], [C] by relevance"
- Captures relative relevance, but limited by context window

Sliding Window Listwise – Handle many documents:

- Rank in overlapping windows, merge results
- Scales to 100+ documents while maintaining comparison quality

Trade-off: Zero-shot/high quality *vs.* higher latency

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

METRICS (1)

Recall@k – *Did we find relevant documents?*

- Fraction of relevant documents retrieved in the top k:

$$\text{Recall@}k = \frac{|\text{relevant} \cap \text{top-}k|}{|\text{relevant}|}$$

- **Intuition:** "Of 10 relevant docs, we found 8" $\rightarrow$ Recall@k = 0.8
- ⚠ Does not measure *order* within the top k

Mean Reciprocal Rank (MRR) – *How fast do we find something useful?*

- Average of $1/\text{rank}$ of the *first* relevant document across queries

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

- **Intuition:** Rank 1 $\rightarrow$ 1.0, Rank 2 $\rightarrow$ 0.5, Rank 10 $\rightarrow$ 0.1
- ⚠ Only rewards *first* hit; ignores further relevant documents

METRICS (2)

Normalized Discounted Cumulative Gain (nDCG@k)

– *How well-ordered is the ranking?*

Key ideas:

- Documents have *graded* relevance (e.g. 0 = irrelevant, 1 = partial, 2 = highly relevant)
- nDCG rewards getting the most relevant one to the top positions
- A log discount penalises relevant docs that appear too low

METRICS (3)

Formula:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)} \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

where rel_i is the relevance grade of the document at rank i, and IDCG@k is the DCG of the *ideal* ranking (best possible ordering).

Example (grades: 2, 1, 0, 2 at ranks 1–4):

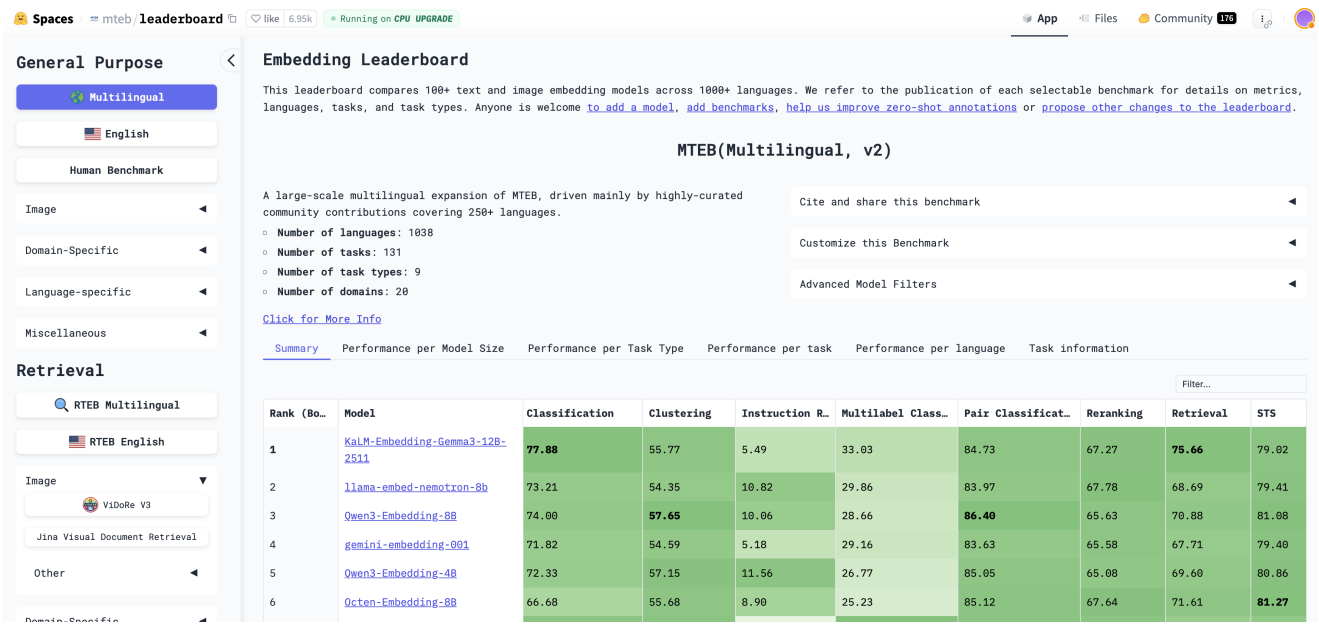
$$\text{DCG@4} = \frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} + \frac{2}{\log_2 5} = 2 + 0.63 + 0 + 0.86 = 3.49$$

Ideal order would be (2, 2, 1, 0):

$$\text{IDCG@4} = 2 + 1.26 + 0.5 + 0 = 3.76, \text{ so nDCG@4} = \frac{3.49}{3.76} \approx \mathbf{0.93}$$

BENCHMARKS (1): MTEB

- Massive Text Embedding Benchmark ▸ Muenninghoff et al. (2023) [15]
- > 8 task types: retrieval, classification, clustering, reranking, STS, etc.
- > 58 datasets across domains and languages



▸ Source: MTEB Leaderboard

Practical Implication: MTEB as reference for embedding models.

BENCHMARKS (2): BEIR

A diverse information retrieval benchmark: ▸ Thakur et al. (2021) [24]

- 18 diverse retrieval datasets
- Tests **zero-shot generalization** across domains
- Includes: MS MARCO, Natural Questions, TREC-COVID,etc.

Why BEIR matters:

- Models often overfit to MS MARCO
- BEIR reveals better robustness
- Key finding: BM25 often competitive on out-of-domain data

Smaller version (for limited compute resources): ▸ Nano-BEIR

QUESTION TYPES: SINGLE- VS. MULTI-HOP

Single-hop question: The answer is contained in *one* document.

Example

"What is the capital of France?" $\rightarrow$ one document suffices

Multi-hop question: The answer requires *chaining* facts across multiple documents.

Example

"Where was the director of Oppenheimer born?"

$\rightarrow$ Doc 1: *Nolan directed Oppenheimer* $\xrightarrow{\text{bridges to}}$ Doc 2: *Nolan was born in London*

Why does this matter for RAG?

- One-Step-Retrieval handles single-hop questions well
- Multi-hop questions *cannot* be answered in a single step
  $\rightarrow$ Unclear what to look for in hop 2 before seeing hop 1

MULTI-HOP QUESTIONS: TYPES

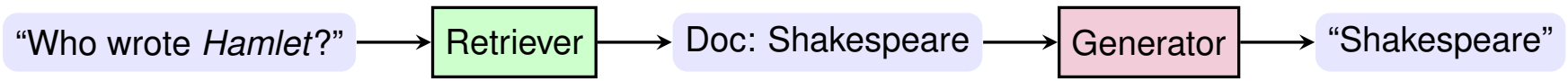
Types of multi-hop reasoning:

- **Bridge:** Answer to sub-question 1 is the *entity* to look up next
  "What country is the birthplace of the inventor of the telephone?"
- **Intersection:** Answer must satisfy two independent conditions
  "Name a Nolan film starring Cillian Murphy"
- **Comparison:** Retrieve facts about two entities, then compare
  "Who is older, Einstein or Bohr?"

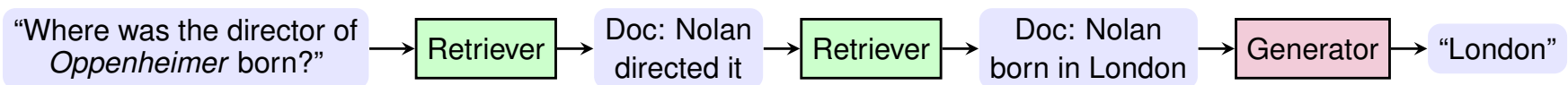
Key challenge: Errors compound – a wrong first-hop document can easily derail subsequent hops.

SINGLE-HOP VS. MULTI-HOP: STRATEGIES (1)

Single-hop: One retrieval step is sufficient to answer the question.



Multi-hop: The answer requires chaining evidence across multiple documents.



SINGLE-HOP VS. MULTI-HOP: STRATEGIES (2)

	Single-hop	**Multi-hop**
Retrieval steps	1	2+ (iterative)
Query	Fixed	Updated after each hop
Complexity	Low	High
Failure modes	Miss + wrong doc	Error propagation
Example dataset	Natural Questions	HotpotQA ▸ Yang et al. (2018) [26]

Multi-hop strategies:

- **Iterative:** Use partial answers to reformulate the query for the next hop
- **Query decomposition:** Break complex question into sub-questions
- **Interleave Chain-of-thought & retrieval** ▸ Trivedi et al. (2023) [25]

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

GROUNDING AND SOURCE ATTRIBUTION

Overview:

Stage	Approach	Key Methods
During Generation	Train/prompt LLM to cite during generating: “According to [1], ...”	▸ Nakano et al. (2022) [16] ▸ Menick et al. (2022) [14]
Reasoning, then generate	First reason about sources and plan answer, then generate grounded text	▸ Slobodkin et al. (2024) [22]
Post-hoc Attribution	Generate first, then retrieve and fact-check each claim and add or fix citations	▸ Gao et al. (2023) [6]

PROMPT DESIGN

How retrieved documents reach the generator matters:

1. **Concatenation (standard):**
   ```
   Context: [doc1] [doc2] ...[docK]
   Question: {q}
   Answer:
   ```
2. **Fusion-in-Decoder (FiD)** ▸ Izacard & Grave (2021) [9]
   Encode each (question, passage) pair *separately* with the encoder, then fuse in decoder
   $\rightarrow$ Scales to many passages without quadratic attention cost
3. **Iterative design:**
   Generate intermediate reasoning steps; each step can trigger a new retrieval $\rightarrow$ cf. *Multi-hop RAG problems*

Practical recommendation:
Fewer, higher-quality passages often beat many noisy ones.

FAITHFULNESS & HALLUCINATION CONTROL

Question: How can we assure that generators do not simply ignore retrieved evidence and fall back on their parametric memory?

Approaches to improve faithfulness:

- **Citation-aware training / prompting:** Ask the model to quote passages inline (e.g. WebGPT, GopherCite)
- **Self-RAG** ▸ Asai et al. (2024) [2]
  Model learns to decide *when* to retrieve and inserts *reflection tokens* (`[Retrieve]`, `[IsRel]`, `[IsSup]`, `[IsUse]`) to self-critique its output

FAITHFULNESS & HALLUCINATION CONTROL

Question: How can we assure that generators do not simply ignore retrieved evidence and fall back on their parametric memory?

Approaches to improve faithfulness:

- **Constraint decoding:** Force the model to generate tokens that appear in evidence
- **Separate verifier:** A second model checks each claim against retrieved passages

Key message: Retrieval quality sets a ceiling; faithfulness techniques help the generator get close to it.

EVALUATING RAG GENERATION

Example of generation metrics (there exist many more):

Metric	Measures	How
Exact Match (EM)	Correctness	Answer = gold exactly?
BERTScore	Semantic similarity	Similarity of answer to gold
Faithfulness	Grounding	Is answer supported by docs?

Practical recommendation:

$\rightarrow$ Evaluate components separately to diagnose failures

Complete Evaluation Frameworks:

- ARES ▸ Saad-Falcon et al. (2024) [20]
- RAGAs ▸ Es et al. (2024) [4]

OUTLINE

Motivation

Retrieval Methods

Reranking

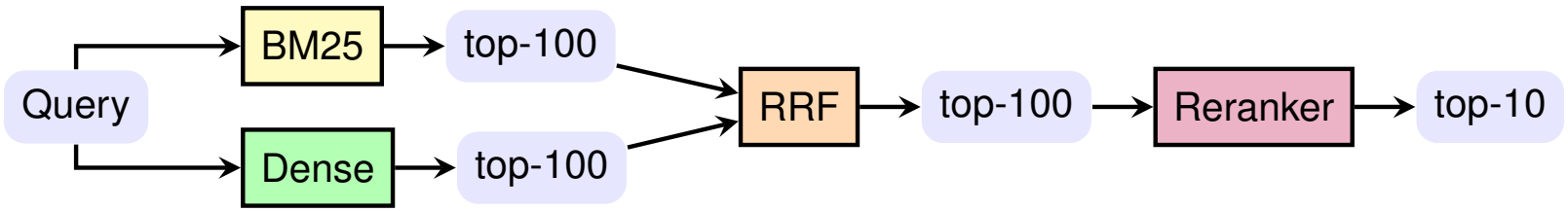
Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

PRACTICAL RETRIEVAL PIPELINE



Reciprocal Rank Fusion: Document rank aggregation algorithm (based on multiple ranking lists, create unified ranking order)

Why hybrid (often) wins:

- BM25 catches exact matches
- Dense embeddings catch semantic matches
- **Complementary failure modes** $\rightarrow$ recall improvement

PRACTICAL RECOMMENDATIONS

1. Understand your data:

- What are typical queries? What is your user interested in?
- Does the LLM have domain knowledge?

2. Start simple, iterate: Many people start with building complex agents.

1. BM25 + basic chunking (baseline)
2. Add hybrid retrieval and reranking
3. Then add query improvement or agentic behavior. This needs careful design for dedicated datasets

3. Monitor: Evaluate each stage individually

OUTLINE

Motivation

Retrieval Methods

Reranking

Evaluation (Retriever)

Evaluation (Generator)

Practical Considerations

Case Study

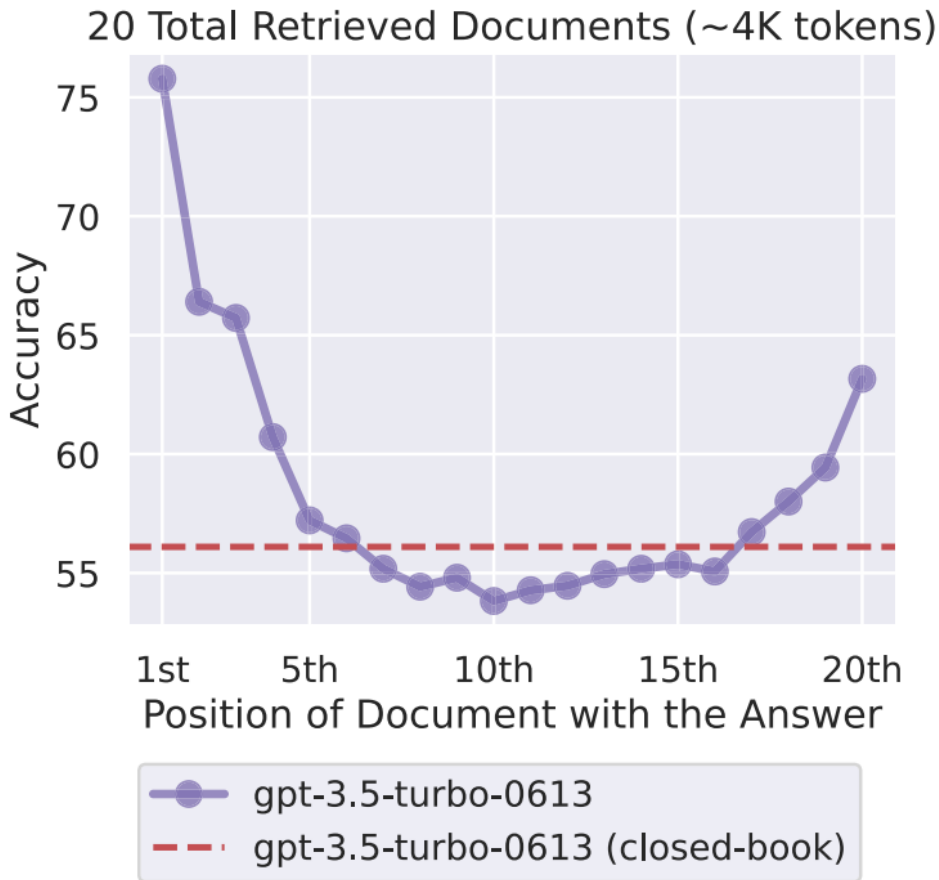
EXAMPLE: LOST IN THE MIDDLE (SETUP)

Question: Do long-context LLMs use information *uniformly* across positions? ▸ Liu et al. (2024) [13]

- *Experimental Lost in the Middle* (LiM) design:
  - Insert one relevant document at varying positions within a long context
  - Surround it with distractor documents of similar form
  - Measure answer accuracy as a function of the relevant document's position
- Tasks: Multi-document QA and key-value retrieval
- This directly probes positional bias in practical long-context usage

EXAMPLE: LOST IN THE MIDDLE (MAIN FINDING)

Consistent pattern across models: **U-shaped performance curve**



▸ Source: [13]

EXAMPLE: LOST IN THE MIDDLE (MAIN FINDING)

Consistent pattern across models: **U-shaped performance curve**

- **Primacy effect:** Evidence at the *beginning* $\rightarrow$ highest accuracy
- **Recency effect:** Evidence at the *end* $\rightarrow$ second-best accuracy
- **Middle degradation:** Accuracy drops substantially when evidence is buried in the middle
- Effect is pronounced even in models with **32k+ context windows**

$\Rightarrow$ Larger context window does *not* imply uniform access to all positions

EXAMPLE: LOST IN THE MIDDLE (MAIN FINDING)

Practical implications for RAG:

- **Rerank before prompt construction:** Put the most relevant chunks first (or last)
- **Limit the number of passages:** Fewer, better passages beat many mediocre ones
- **Don’t (always) assume "more context = better"**

⚠ This behaviour has changed/might change with newer models ⚠

LOST IN THE MIDDLE: WHY DOES THIS HAPPEN?

Hypothesis 1 – Attention sink / primacy bias:

- Autoregressive LLMs pay disproportionate attention to early tokens (BOS effect)
- Empirically confirmed: attention weights decay toward the middle

Hypothesis 2 – Training data bias:

- Instruction-tuning data typically has the answer near the beginning or end of provided context $\rightarrow$ Model learns a positional shortcut

LOST IN THE MIDDLE: WHY DOES THIS HAPPEN?

Connections to retrieval:

Problem	Mitigation
Evidence buried in middle	Reranker-ordered prompts
Too many passages	Top-k truncation ($k \leq$ 5–10)
Position-agnostic eval	Shuffle eval (test robustness)

Open questions:

- Does instruction tuning on position-diverse data help?
  ▸ Liu et al. (2024) [13]
- What is the role of positional encoding? ▸ Zehle & Aßenmacher (2026) [27]

REFERENCES I

[1] Abdelrahman Abdallah, Bhawna Piryani, Jamshid Mozafari, Mohammed Ali, and Adam Jatowt. How good are LLM-based rerankers? an empirical analysis of state-of-the-art reranking models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5693–5709, Suzhou, China, November 2025. Association for Computational Linguistics.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, volume 2024, pages 9112–9141, 2024.

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. In *arXiv preprint, https://arxiv.org/abs/2401.08281*, 2024.

[4] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In Nikolaos Aletras and Orphee De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, March 2024. Association for Computational Linguistics.

[5] Mohsen Fayyaz, Ali Modarressi, Hinrich Schuetze, and Nanyun Peng. Collapse of dense retrievers: Short, early, and literal biases outranking factual evidence. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9136–9152, Vienna, Austria, July 2025. Association for Computational Linguistics.

[6] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. RARR: Researching and revising what language models say, using language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508, Toronto, Canada, July 2023. Association for Computational Linguistics.

[7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. In *arXiv preprint, https://arxiv.org/abs/2312.10997*, 2023.

[8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM transactions on information systems*, 43(2):1–55, 2025.

REFERENCES II

[9] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty, editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, Online, April 2021. Association for Computational Linguistics.

[10] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics.

[11] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '20, page 39–48, New York, NY, USA, 2020. Association for Computing Machinery.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[14] Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, Mia Glaese, Susannah Young, Lucy Campbell-Gillingham, Geoffrey Irving, and Nat McAleese. Teaching language models to support answers with verified quotes. In *arXiv preprint, https://arxiv.org/abs/2203.11147*, 2022.

[15] Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In Andreas Vlachos and Isabelle Augenstein, editors, *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics.

[16] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. In *arXiv preprint, https://arxiv.org/abs/2112.09332*, 2022.

REFERENCES III

[17] Rodrigo Nogueira and Kyunghyun Cho. Passage Re-ranking with BERT. In *arXiv preprint, https://arxiv.org/abs/1901.04085*, 2020.

[18] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, November 2019. Association for Computational Linguistics.

[19] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[20] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 338–354, Mexico City, Mexico, June 2024. Association for Computational Linguistics.

[21] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz, editors, *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, Seattle, United States, July 2022. Association for Computational Linguistics.

[22] Aviv Slobodkin, Eran Hirsch, Arie Cattan, Tal Schuster, and Ido Dagan. Attribute first, then generate: Locally-attributable grounded text generation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3309–3344, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[23] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937, Singapore, December 2023. Association for Computational Linguistics.

REFERENCES IV

[24] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021.

[25] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037, Toronto, Canada, July 2023. Association for Computational Linguistics.

[26] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October-November 2018. Association for Computational Linguistics.

[27] Tom Zehle and Matthias Aßenmacher. Can calibration of positional encodings enhance long context utilization? In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Findings of the Association for Computational Linguistics: EACL 2026*, pages 2268–2280, Rabat, Morocco, March 2026. Association for Computational Linguistics.